
Does the Concepts TS Improve on C++17?

One of the primary goals of the [C++ Concepts TS](#) is to provide better template error messages. Using the reference implementations provided by [GCC 7.1's Concepts](#) and [concept-enabled Ranges](#), this paper evaluates the error messages for a few simple STL use cases. The results are surprising, and not encouraging.

The Concepts TS is also intended to simplify Generic Programming, but in practice, concept requirements are far from simple. The [Ranges TS](#), whose concepts depend on metaprogramming techniques and other arcana, demonstrates the challenges of making concepts work in the real world.

Finally, C++17 already has the tools to express requirements on templates, check valid syntax, and compose those checks into reusable entities, providing most of the features of concepts without new language extensions. The Concepts TS adds significant complexity and little value to C++17. Until demonstrable benefits outweigh the costs, it should not be merged.

Contents

- [Error Messages with Concepts and Ranges](#)
 - [More sorting](#)
- [Generic Programming with Concepts](#)
 - [Real-world concepts](#)
- [C++17 Already Has the Tools for Generic Programming](#)
 - [Requires expressions](#)
 - [Concept-based overloading](#)
- [Conclusion](#)

Error Messages with Concepts and Ranges

Using [GCC 7.1's implementation of Concepts](#) and [Casey Carter's reference implementation of the Ranges TS](#), let's take a canonical example (drawn from [here](#) but used everywhere) where the `sort` algorithm is called on a `list`:

```
std::list<int> l = {2, 1, 3};
std::experimental::ranges::sort(l.begin(), l.end());
```

The complete diagnostic is 99 lines, with 35 notes, and is available in [this gist](#). It begins by noting the `sort` candidate that failed:

```
note: candidate: I std::experimental::ranges::v1::sort(I, S, Comp, Proj) [with I = std::_List_iterator<int>, S = std::_List_iterator<int>, Comp = std::less<int>, Proj = std::identity<int>]
I sort(I first, S sent, Comp comp = Comp{}, Proj proj = Proj{})
```

and that the constraints of this candidate are unsatisfied:

```
note: constraints not satisfied
```

Next is a note that indicates it was the `RandomAccessIterator` requirement that failed:

```
note: within 'template<class I> concept const bool std::experimental::ranges::v1::RandomAccessIterator(I)'
```

Anyone that understands the concepts involved can stop here. However, the diagnostic continues for another 30+ notes, highlighting various syntactic failures, e.g., regarding iterator tags:

```
note: within 'template<class T, class U> concept const bool std::experimental::ranges::v1::RandomAccessIteratorTag(T, U)'
```

```
note: 'std::experimental::ranges::v1::random_access_iterator_tag' is not a base of 'std::experimental::ranges::v1::random_access_iterator_tag'
```

More sorting

Let's consider a similar example ([gist here](#)):

```
void f2(std::vector<MyType> &v) {
    std::experimental::ranges::sort(v.begin(), v.end());
}
```

Here, GCC again identifies the candidate template and that the constraints are unsatisfied:

```
note: candidate: I std::experimental::ranges::v1::sort(I, S, Comp, Proj) [with I = __gnu_cxx::__normal_iterator<MyType*>, S = __gnu_cxx::__normal_iterator<MyType*>, Comp = std::less<MyType>, Proj = std::identity<MyType>]
I sort(I first, S sent, Comp comp = Comp{}, Proj proj = Proj{})
    ^~~~
include/stl2/detail/algorithm/sort.hpp:31:4: note: constraints not satisfied
```

followed by the concept constraint that failed:

```
note: within 'template<class I, class R, class P> concept const bool std::experimental::ra
```

`Sortable` doesn't tell us enough about what went wrong, so we follow the sequence of notes:

```
note: within 'template<class F, class I1, class I2> concept const bool std::experimental::ra
note: within 'template<class R, class T, class U> concept const bool std::experimental::ra

.
. (skipped 29 notes)
.

note: ... and 5 more constraint errors not shown
```

The actual problem is that `MyType` didn't provide an operator `<`, which didn't show up in any of the 85+ lines of error messages produced by GCC.

One of the main selling points of the Concepts TS is improved template error messages, but experience with simple STL examples shows quite the opposite: the diagnostics are verbose, dumping a deep backtrace of concept references, while not providing critical information about what went wrong to the user. In fact, the error message provided by the same compiler in C++17 mode is actually better than what is provided by concepts, because it almost immediately states the lack of `<`:

```
/opt/gcc/include/c++/7.1.0/bits/predefined_ops.h:43:23: error: no match for 'operator<' (c
    { return *__it1 < *__it2; }
          ~~~~~^~~~~~
```

Generic Programming with Concepts

Let's look at concepts from the other side, and add requirements to an existing generic algorithm. Sticking with `sort`, its most prominent requirement is the need for random access iterators:

```
template <class Iter>
    requires RandomAccessIterator<Iter>
void sort(Iter first, Iter last) { ... }
```

This is a straightforward change, and will trigger a concept-based diagnostic if the user calls us with

`list` iterators. However, users will find that it's insufficient: sorting a container with a type that lacks `<`

will still produce a diagnostic from inside the template's implementation. Fortunately, that's easy to fix with another requirement:

```
template <class Iter>
    requires RandomAccessIterator<Iter> && StrictWeakOrder<value_type_t<Iter>>
void sort(Iter first, Iter last) { ... }
```

That's closer, but users will eventually discover that we still haven't covered `const` :

```
void f(const std::vector<int> &v) {
    sort(v.begin(), v.end()); // traditional template backtrace
}
```

Perhaps this is a natural back-and-forth when adopting concepts, but where does it lead? Do concepts make generic code simpler to write?

Real-world concepts

To find out, let's go back to the Ranges TS for the requirements on `sort` :

```
template <RandomAccessIterator I, Sentinel<I> S, class Comp = less<>,
        class Proj = identity>
    requires Sortable<I, Comp, Proj>()
I sort(I first, S last, Comp comp = Comp{}, Proj proj = Proj{});
```

This looks simple, but somebody actually had to write `Sortable` . Let's take a look at what's involved:

```

template <class I, class R = less<>, class P = identity>
concept bool Sortable() {
    return Permutable<I>() &&
        IndirectStrictWeakOrder<R, projected<I, P>>();
}

template <class I> concept bool Permutable() { ... }

template <class F, class I1, class I2 = I1>
concept bool IndirectStrictWeakOrder() {
    return Readable<I1>() && Readable<I2>() &&
        CopyConstructible<F>() &&
        StrictWeakOrder<F&, value_type_t<I1>&, value_type_t<I2>&>() &&
        StrictWeakOrder<F&, value_type_t<I1>&, reference_t<I2>>>() &&
        StrictWeakOrder<F&, reference_t<I1>, value_type_t<I2>&>() &&
        StrictWeakOrder<F&, reference_t<I1>, reference_t<I2>>>() &&
        StrictWeakOrder<F&, iter_common_reference_t<I1>, iter_common_reference_t<I2>>>() &&
}

```

By the time we get to a lower-level concept such as `IndirectStrictWeakOrder`, we see the complexity required to express simple notions correctly. Still, we haven't gotten to the key constraint that allows ordering, so we'll keep going:

```

template <class R, class T, class U>
concept bool StrictWeakOrder() {
    return Relation<R, T, U>();
}

template <class R, class T>
concept bool Relation() {
    return Predicate<R, T, T>();
}

template <class R, class T, class U>
concept bool Relation() {
    return Relation<R, T>() &&
        Relation<R, U>() &&
        CommonReference<const T&, const U&>() &&
        Relation<R,
            common_reference_t<const T&, const U&>>() &&
        Predicate<R, T, U>() &&
        Predicate<R, U, T>();
}

template <class F, class... Args>
concept bool Predicate() {
    return RegularInvocable<F, Args...>() &&
        Boolean<result_of_t<F&&(Args&&...)>>()>();
}

```

Here, we see what is effectively a syntactic requirement: the result of calling `F` with the given arguments (`Args`) must satisfy the `Boolean` concept. `Boolean` is specified as:

```

template <class B>
concept bool Boolean() {
    return MoveConstructible<B>() &&
        requires(const B b1, const B b2, const bool a) {
            bool(b1);
            { b1 } -> bool;
            bool(!b1);
            { !b1 } -> bool;
            { b1 && b2 } -> Same<bool>;
            { b1 && a } -> Same<bool>;
            { a && b1 } -> Same<bool>;
            { b1 || b2 } -> Same<bool>;
            { b1 || a } -> Same<bool>;
            { a || b1 } -> Same<bool>;
            { b1 == b2 } -> bool;
            { b1 != b2 } -> bool;
            { b1 == a } -> bool;
            { a == b1 } -> bool;
            { b1 != a } -> bool;
            { a != b1 } -> bool;
        };
};

```

This is quite a large concept, because it is trying to model all of the different ways in which a `Boolean` type can be correctly used. Interestingly, some of the requirements of `Boolean` constrain the result to `Same<bool>` (to mean "exactly `bool`") while others use `bool` (to mean "implicitly convertible to `bool`"). The distinction is subtle, requiring fairly deep knowledge of C++.

One can imagine a simple definition for `Sortable`, but the Ranges TS illustrates that simple concepts aren't sufficient: to actually capture the intended behavior of a C++ template, one needs to expose all of the complexity of the language. The concepts above are not simple to understand, even for an expert, and they're not unique in that regard: browsing through the Ranges TS, many of the concepts have extremely complicated definitions with subtle uses of `Same`, `reference_t`, and helper concepts like `Boolean` and `CommonReference`. Writing new concepts requires deep knowledge of not only the new features in the Concepts TS, but of several other difficult parts of C++, including overload resolution and implicit conversions.

C++17 Already Has the Tools for Generic Programming

The Concepts TS is providing tools for generic programming, but those tools already exist in C++. For example, `std::enable_if` expresses the requirements of a template. Let's turn that `sort` example into valid C++17:

```
template <class Iter, class = std::enable_if_t<
    RandomAccessIterator<Iter> && LessThanComparable<value_type_t<Iter>>>>
void sort(first, Iter last) { ... }
```

The "concepts" here are `constexpr` variable templates using existing facilities. A simple form of `RandomAccessIterator` checks the iterator category (the iterator category is still used for this purpose in the Ranges TS):

```
template <class Iter>
constexpr bool RandomAccessIterator =
    std::is_convertible<iterator_category_t<Iter>,
        std::random_access_iterator_tag>::value;
```

and `LessThanComparable` uses existing techniques for checking valid expressions:

```
template <class T, class = void>
constexpr bool LessThanComparable = false;

template <class T>
constexpr bool LessThanComparable<T, std::void_t<
    decltype(static_cast<bool>(std::declval<T>() < std::declval<T>()))>> = true;
```

C++ template libraries already use these techniques to state their template requirements. For example, [ranges-v3](#), which is a C++11/14 implementation for the Ranges TS, makes extensive use of `enable_if` and concepts-like checking.

Requires expressions

The syntactic checks of the previous section were drastically simplified. However, more involved syntactic checks on par with the Ranges TS can be implemented already in C++17. For example, here is a C++17 definition of the `StrictTotallyOrdered` concept from the Ranges TS:


```

template <class T, class U = T>
constexpr bool StrictTotallyOrdered =
    CommonReference<const T&, const U&> &&
    StrictTotallyOrdered<T> &&
    StrictTotallyOrdered<U> &&
    StrictTotallyOrdered<
        std::remove_cv_t<std::remove_reference_t<
            std::common_reference_or_void_t<const T&, const U&>>>>() &&
    EqualityComparable<T, U> &&
    requires<const T, const U>([&](auto&& t, auto&& u) -> std::enable_if_t<
        Boolean<decltype(t < u)> &&
        Boolean<decltype(t > u)> &&
        Boolean<decltype(t <= u)> &&
        Boolean<decltype(t >= u)> &&
        Boolean<decltype(u < t)> &&
        Boolean<decltype(u > t)> &&
        Boolean<decltype(u <= t)> &&
        Boolean<decltype(u >= t)>> {}]);

template <class T>
constexpr bool StrictTotallyOrdered<T> =
    EqualityComparable<T> &&
    requires<const T, const T>([&](auto&& a, auto&& b) -> std::enable_if_t<
        Boolean<decltype(a < b)> &&
        Boolean<decltype(a > b)> &&
        Boolean<decltype(a <= b)> &&
        Boolean<decltype(a >= b)>> {}]);

```

This defines the concepts `StrictTotallyOrdered<T, U>` and `StrictTotallyOrdered<T>` as `constexpr bool` variables, relying on previously-defined concepts [as described at cppreference](#) and [demoed in this gist](#).

Focusing on the single-argument case, `StrictTotallyOrdered<T>` refines `EqualityComparable<T>`, and requires that the return type of the relational operators on `const T` conform to the `Boolean` concept.

The syntactic logic is handled by the `requires` function (inspired by [hana::is_valid](#), and not so different from [std::experimental::is_detected](#)). `requires` takes a type list as template parameters and a generic lambda as an argument, and returns `true` iff the lambda can be called with that type list. The return type of the lambda is a playground for testing arbitrary C++ syntax inside `decltype`, and here was used to check for concept conformance.

Here are a few examples to demonstrate its use more broadly:

```

// T has operator++
requires<T>([](auto&& x) -> decltype(++x) {})

// T has operator++ and return type is T&
requires<T>([](auto&& x) -> std::enable_if_t<Same<decltype(++x), T&>> {})
requires<T>([](auto&& x) -> std::enable_if_t<
    std::is_same_v<decltype(++x), T&>> {})) // equivalent

// A bool can be constructed from const T
requires<const T>([](auto&& x) -> decltype(bool(x))) {}
requires<const T>([](auto&& x) -> std::enable_if_t<
    std::is_constructible_v<bool, decltype(x)>> {})) // equivalent

// Return type of operator&& on const T is a Boolean
requires<const T>([](auto&& x) -> std::enable_if_t<
    Boolean<decltype(x && x)>> {}))

// Return type of operator== on const T is implicitly convertible to bool
requires<const T>([](auto&& x) -> std::enable_if_t<
    ConvertibleTo<decltype(x == x), bool>> {}))

// Return type of operator! on const T is bool.
requires<const T>([](auto&& x) -> std::enable_if_t<
    Same<decltype(!x), bool>> {}))

// Return type of operator+= on T and const U is T&.
requires<T, const U>([](auto&& t, auto&& u) -> std::enable_if_t<
    Same<decltype(t += u), T&>> {}))

```

The reference implementation is straightforward.

```

namespace detail {
template <class F, class... Args,
        class = decltype(std::declval<F&&>()(std::declval<Args&&>()...))>
constexpr bool requires_impl(int) { return true; }
template <class F, class... Args>
constexpr bool requires_impl(...) { return false; }
} // namespace detail

template <class... Args, class F>
constexpr bool requires(F&&) {
    return detail::requires_impl<F&&, Args&&...>(int{});
}

```

Concept-based overloading

Another feature of the Concepts TS is concept-based overloading, which allows multiple versions of the same template that differ only based on their requirements. A typical example is `std::advance`:

```
template <class Iter>
    requires InputIterator<Iter>
void advance(It& i, difference_type_t<Iter> n) {
    while (n > 0) { ++i; --n; }
}

template <class Iter>
    requires BidirectionalIterator<Iter>
void advance(It& i, difference_type_t<Iter> n) {
    while (n < 0) { --i; ++n; }
    while (n > 0) { ++i; --n; }
}

template <class Iter>
    requires RandomAccessIterator<Iter>
void advance(It& i, difference_type_t<Iter> n) {
    i += n;
}
```

A call to `advance` will pick the most specialized algorithm. C++17 provides a more concise, direct way of accomplishing the same thing via `constexpr-if`:

```
template <class Iter, class = std::enable_if_t<
    InputIterator<Iter>>>
void advance(It& i, difference_type_t<Iter> n) {
    if constexpr (RandomAccessIterator<Iter>) {
        i += n;
    } else if constexpr (BidirectionalIterator<Iter>) {
        while (n < 0) { --i; ++n; }
        while (n > 0) { ++i; --n; }
    } else {
        while (n > 0) { ++i; --n; }
    }
}
```

The `if constexpr` formulation is simpler than the one using concept-based overloading. It only presents a single operation to the user via a single signature, which better matches the intent: there is *one* `advance` operation provided, and it happens that it will provide more efficient behavior with some types. The implementation is also much simpler, because it reads like normal executable code with easy-to-understand control flow.

There are some (rare) cases where a third-party might want to introduce a different implementation based on new concepts, but these can be addressed by other [well-known techniques](#) for overloading. The vast

majority of uses of concept-based overloading are better addressed via `constexpr-if`.

Conclusion

The introduction of new features can simplify the C++ language, and `constexpr` is a fine example. Before `constexpr`, computing constant values involved arcane template metaprogramming that baffled many users. `constexpr` made such computations drastically simpler, by extended the capabilities of otherwise-ordinary functions in intuitive ways, opening up compile-time computation to a much larger audience. Similarly, lambdas opened up functional programming idioms to a far larger audience by making function objects more convenient and ubiquitous.

The Concepts TS does not simplify Generic Programming. The experience with GCC and the reference implementation of the Ranges TS raises serious concerns about whether they can improve the experience of using template libraries. The Ranges TS itself illustrates how the language complexity leaks through concepts and into the user experience, and demonstrates that real-world concepts are not simple or easy to write or use.

In the 14 years since this approach to concept support was first proposed, C++ has evolved significantly, with the introduction of `constexpr`, variable templates, `constexpr-if`, the expansion of SFINAE, and the invention of numerous new template techniques, which together provide the tools that developers are already using to express the ideas of generic programming. Any language feature added to support these idioms should significantly improve on the results we can get today.

The Concepts TS does not appear to do so.